



Universidad Nacional de San Luis

Departamento de Informática

Trabajo de Laboratorio
Sistemas de Tiempo Real

Semáforo Inteligente

Ayelen Yanina Humerez Colque

Facundo Nicolás Farias Lozano

Giuliano Agustin Carbone

Juan Cruz Paez

Profesores:

Prof. Alejandro Sánchez

Ingeniería en Informática

2026

Actividades

Consideraciones/Suposiciones

El modelado de todo el proyecto se realizó bajo las siguientes suposiciones acerca del comportamiento del semáforo de calle:

- Los cambios de estado se dan como greenyellow, yellowred, redgreen.
- Los semáforos de una misma calle están sincronizados, por lo tanto, solo se controla el comportamiento de los semáforos en la calle Norte-Sur y los semáforos de la calle Este-Oeste.

Ejercicio 1

Consigna: Modelo base cíclico fijo. Implementar en NuSMV un modelo donde los semáforos cambian de fase únicamente basados en temporizadores (sin considerar demanda). Elegir valores adecuados para $T_{verde} = 10$ ticks, $T_{amarillo} = 3$ ticks. Verificar si las propiedades se cumplen.

Para resolver esta consigna, además de las suposiciones generales para todos los modelos, también se consideraron la siguientes suposiciones:

1. El tiempo máximo TM es la suma de los tiempos T_{green} y T_{yellow} . Entonces un ciclo dura $TM = 10 + 3 = 13$ ticks.
2. Nunca los semáforos de ambas calles estarán en rojo simultáneamente.
3. Un auto, al cruzar el semáforo, permanece en la misma calle.

```
1  MODULE semaforo(cross, id, TM, TYellow, TGreen)
2  VAR
3  state : {red, yellow, green};
4  time : 0..TM;
5  ASSIGN
6  init(state) := case
7      cross = id : green;
8      TRUE: red;
9  esac;
10 init(time) := case
11     cross = id : 0;
12     cross != id : TM;
```

```

13     esac;
14     next(state) := case
15         state = green & cross = id    : yellow;
16         state = yellow & cross = id    : red;
17         state = red & next(cross) = id : green;
18         TRUE: red;
19     esac;
20
21     next(time) := case
22         next(state) = green : 0;
23         next(state) = yellow : TGreen;
24         next(state) = red   : TM;
25     esac;
26
27
28     MODULE main
29     VAR
30         semaforo_NS : semaforo(cross, ns, TM, TYellow, TGreen);
31         semaforo_OE : semaforo(cross, oe, TM, TYellow, TGreen);
32         cross : {ns, oe};
33
34     ASSIGN
35         init(cross) := {ns, oe};
36         next(cross) := case
37             cross = ns & semaforo_NS.state = yellow : oe;
38             cross = oe & semaforo_OE.state = yellow : ns;
39             TRUE : cross;
40         esac;
41
42     DEFINE
43         TYellow := 3;
44         TGreen  := 10;
45         TM := TYellow + TGreen;  -- El tiempo total del ciclo
46
47     -- phi 1: Los semaforos de direcciones cruzadas no pueden estar
48     -- en green simultaneamente.
49     LTLSPEC NAME ambosSemaforosIguales := G(semaforo_NS.state !=
50     semaforo_OE.state); --verifica que ambos semaforos no sean rojos
51
52     -- phi 2: El yellow debe durar exactamente TYellow y no debe
53     saltarse.
54     LTLSPEC NAME spec2 := G(semaforo_NS.time = TGreen -> X(
55     semaforo_NS.time = TM) & G (semaforo_NS.state = green -> X(
56     semaforo_NS.state = yellow))) & G(semaforo_OE.time = TGreen -> X(
57     semaforo_OE.time = TM) & G (semaforo_OE.state = green -> X(
58     semaforo_OE.state = yellow)));
59
60     LTLSPEC NAME spec2_2 := G(semaforo_NS.state = yellow ->
61     semaforo_NS.time = TGreen & X(semaforo_NS.state = red ->
62     semaforo_NS.time = TM)); --Sabemos que TM es TYellow -- phi 4:
63     El tiempo de green no debe exceder un maximo Tmax (ej. 30
64     segundos).
65
66     -- Q4
67     LTLSPEC NAME noExcede := G(TGreen < TM);

```

```

57
58     -- phi 5: Siempre es posible que algun semaforo este en green
59     CTLSPEC NAME posibleVerde := AG( EF (semaforo_NS.state = green
    | semaforo_OE.state = green));

```

```

ProyectoSTR-Semaforo docs/report > nusmv -int Ejer1.smv
*** This is NuSMV 2.7.1 (compiled on (unknown))
*** Enabled addons are: compass
*** For more information on NuSMV see <http://nusmv.fbk.eu>
*** or email to <nusmv-users@list.fbk.eu>.
*** Please report bugs to <Please report bugs to <nusmv-users@fbk.eu>>

*** Copyright (c) 2010-2026, Fondazione Bruno Kessler

*** This version of NuSMV is linked to the CUDD library version 2.4.1
*** Copyright (c) 1995-2004, Regents of the University of Colorado

NuSMV > go
NuSMV > check_tltspec
-- specification G TGreen < TM is true
-- specification G semaforo_NS.state != semaforo_OE.state is true
-- specification G (semaforo_NS.state = yellow -> (semaforo_NS.time = TGreen & X (semaforo_NS.state = red -> semaforo_NS.time = TM))) is true
-- specification ( G (semaforo_NS.time = TGreen -> ( X semaforo_NS.time = TM & G (semaforo_NS.state = green -> X semaforo_NS.state = yellow)))
& G (semaforo_OE.time = TGreen -> ( X semaforo_OE.time = TM & G (semaforo_OE.state = green -> X semaforo_OE.state = yellow)))) is true
NuSMV > check_ctlspec
-- specification AG (EF (semaforo_NS.state = green | semaforo_OE.state = green)) is true

```

Figura 1: Resultados de verificación para el Ejercicio 1.

Ejercicio 2

Consigna: Modificar el modelo para que el semáforo se mantenga en verde si hay demanda en su mismo sentido, hasta un máximo de $T_{max} = 15$ ticks. Si no hay demanda en el sentido actual pero sí en el otro, el cambio de fase ocurre inmediatamente (respetando el amarillo). Verificar las propiedades nuevamente y si alguna falla, explicar por qué.

Para esta actividad, se mantuvo la suposición 2 del modelo base, se cambió el nombre de la variable T_{green} por $T_{greenMin}$ y se agregó la variable $T_{greenMax} = 15$. Se asumió lo siguiente:

- El tiempo máximo TM será la suma de $T_{greenMax}$ y T_{yellow} . Tiempo en el que el semáforo está en amarillo ahora crece por ticks, para eso se agregó dentro del módulo semaforo una variable timeyellow que se incrementa en 1 por cada transición mientras sea menor que T_{yellow} .
- Como el tiempo en amarillo cambia, ya no basta con colocar en la transición de cross que cuando el semaforo este en amarillo se realiza el cambio, sino que ahora se debe tener en cuenta que el semaforo estuvo en amarillo un intervalo de tiempo igual a T_{yellow} .

```

1 MODULE semaforo(cross, id, TM, TGreenMin, TYellow, TGreenMax)
2 VAR
3     state : {red, yellow, green};
4     time : 0..TM;
5     demand : boolean;
6     timeyellow : 0..TYellow;
7
8 ASSIGN

```

```

9  init(state) := case
10      cross = id : green;
11      TRUE: red;
12  esac;
13
14  init(time) := case
15      cross = id : 0;
16      cross != id : TM;
17  esac;
18
19  init(timeyellow) := 0;
20
21  next(state) := case
22      next(cross) = id: case --Si el semaforo tiene el turno
23          state = green : case --Si actualmente esta en verde
24              next(time) < TGreenMin : green; --Que siga en verde si
aun no ha pasado un tiempo minimo
25              demand & next(time) < TGreenMax : green; --Si paso el
tiempo minimo entonces que siga siempre y cuando no pase de un
tiempo maximo
26              TRUE : yellow; --Si llego al tiempo maximo o no hay
demanda, entonces que pase a amarillo
27          esac;
28          state = yellow : case --Si actualmente esta en amarillo
29              next(timeyellow) < TYellow : yellow; --Que siga en
amarillo mientras no haya superado los 3 segundos
30              TRUE: red; --Una vez superado llegue a los 3 segundos,
entonces que pase a rojo
31          esac;
32          state = red : green; --Si es su turno y esta rojo, entonces
significa que ahora debe cambiar a verde
33      esac;
34      TRUE : red; --Si no es su turno, entonces debe mantenerse en
rojo
35  esac;
36
37  next(time) := case
38      next(cross) = id : case --Si es su turno
39          state = red: 0; --Si esta en rojo, eso significa que hay
que empezar el semaforo
40          time < TM: time + 1; --En otro caso, hay que incrementarlo
41          TRUE : TM; --Cuando llegue a TM, que se mantenga ahi
42      esac;
43      TRUE: TM; --Si no es su turno, entonces dejarlo en el tiempo
maximo
44  esac;
45
46  next(demand) := case
47
48      next(state) = green | next(state) = yellow : {TRUE, FALSE}; --
Si esta en verde o amarillo, entonces puede seguir la demanda o
apagarse
49      demand = FALSE : {TRUE, FALSE}; --Si no hay demanda, en
cualquier momento puede llegar autos o seguir sin demanda
50      TRUE : TRUE; --Si el siguiente estado no es verde y hay demanda

```

```

, entonces esta no va a desaparecer
51 esac;
52
53 next(timeyellow) := case
54     state = yellow & timeyellow < TYellow : timeyellow + 1;
55     TRUE : 0;
56 esac;
57
58 MODULE main
59 VAR
60     cross : {ns, oe};
61     semaforo_NS : semaforo(cross, ns, TM, TGreenMin, TYellow,
62                             TGreenMax);
63     semaforo_OE : semaforo(cross, oe, TM, TGreenMin, TYellow,
64                             TGreenMax);
65
66 ASSIGN
67     init(cross) := {ns, oe};
68     next(cross) := case
69         cross = ns & semaforo_NS.state = yellow & next(semaforo_NS.
70 timeyellow) = TYellow : oe; --Si el semaforo esta en amarillo y
71 ya termino su tiempo, ahora le toca empezar al otro semaforo
72         cross = oe & semaforo_OE.state = yellow & next(semaforo_OE.
73 timeyellow) = TYellow : ns; --Si el semaforo esta en amarillo y
74 ya termino su tiempo, ahora le toca empezar al otro semaforo
75         TRUE : cross;
76     esac;
77
78 DEFINE
79     TYellow := 3;
80     TGreenMax := 15;
81     TGreenMin := 10;
82     TM := TYellow + TGreenMax; -- El tiempo total del ciclo
83
84 -- phi 1: Los semaforos de direcciones cruzadas no pueden estar en
85 green simultaneamente.
86 LTLSPEC NAME ambosSemaforosIguales := G(semaforo_NS.state !=
87 semaforo_OE.state); -- Verifica que ambos semaforos no sean
88 rojos
89
90 -- phi 2: El yellow debe durar exactamente TYellow y no debe
91 saltarse.
92 LTLSPEC NAME spec2 := G((semaforo_NS.time >= TGreenMin &
93 semaforo_NS.state = yellow) -> X(semaforo_NS.time <= TM &
94 semaforo_NS.time >= (TYellow + TGreenMin))) | G((semaforo_NS.
95 time >= TGreenMin & semaforo_NS.state = yellow) -> X(semaforo_NS
96 .time <= TM & semaforo_NS.time >= (TYellow + TGreenMax)))
97 LTLSPEC NAME spec2 := G(F(semaforo_NS.timeyellow = TYellow) & F(
98 semaforo_OE.timeyellow = TYellow))
99
100 -- phi 3: Si hay vehiculos esperando en una direccion y el semaforo
101 esta rojo, eventualmente se pondra verde.
102 LTLSPEC NAME demandaYRojo := G((semaforo_NS.demand & semaforo_NS.
103 state = red -> F (semaforo_NS.state = green)) & (semaforo_OE.
104 demand & semaforo_OE.state = red -> F (semaforo_OE.state = green
105 )));

```

```

86
87 -- phi 4: El tiempo de verde no debe exceder un maximo Tmax (ej. 30
    segundos).
88 LTLSPEC NAME noExcede := G(!(semaforo_NS.state = green &
    semaforo_NS.time = TGreenMax) & !(semaforo_OE.state = green &
    semaforo_OE.time = TGreenMax));
89
90 -- phi 5: Siempre es posible que algun semaforo este en green
91 CTLSPEC NAME posibleVerde := AG( EF (semaforo_NS.state = green |
    semaforo_OE.state = green));

```

Listing 1: Modelo con demanda y extensión inteligente

```

ProyectoSTR-Semaforo docs/report ? * nusmv -int Ejer2.smv
*** This is NuSMV 2.7.1 (compiled on (unknown))
*** Enabled addons are: compass
*** For more information on NuSMV see <http://nusmv.fbk.eu>
*** or email to <nusmv-users@list.fbk.eu>.
*** Please report bugs to <Please report bugs to <nusmv-users@fbk.eu>>

*** Copyright (c) 2010-2026, Fondazione Bruno Kessler

*** This version of NuSMV is linked to the CUDD library version 2.4.1
*** Copyright (c) 1995-2004, Regents of the University of Colorado

NuSMV > go
NuSMV > check_ltlspec
-- specification G TGreen < TM is true
-- specification G semaforo_NS.state != semaforo_OE.state is true
-- specification G (semaforo_NS.state = yellow -> (semaforo_NS.time = TGreen & X (semaforo_NS.state = red -> semaforo_NS.time = TM))) is true
-- specification ( G (semaforo_NS.time = TGreen -> ( X semaforo_NS.time = TM & G (semaforo_NS.state = green -> X semaforo_NS.state = yellow)))
& G (semaforo_OE.time = TGreen -> ( X semaforo_OE.time = TM & G (semaforo_OE.state = green -> X semaforo_OE.state = yellow)))) is true
NuSMV > check_ctlspec
-- specification AG (EF (semaforo_NS.state = green | semaforo_OE.state = green)) is true

```

Figura 2: Resultados de verificación para el Ejercicio 2.

Ejercicio 3

Consigna: Modificar el modelo para poder verificar que el tiempo máximo de espera para un vehículo (desde que llega hasta que obtiene verde) no supere $L = 20$ ticks. Especificar esta propiedad en CTL y verificarla.

En esta actividad, además de las suposiciones base, también se asumió lo siguiente:

1. Como en todo momento hay un semaforo en verde o amarillo, solo contabilizamos el tiempo que lleva esperando un auto en el otro semaforo. Es decir, se cuentan cuantos ticks lleva la variable “demand” en true en el semáforo que no tiene aún el turno.

```

1 MODULE semaforo(cross, id, TM, TGreenMin, TYellow, TGreenMax)
2 VAR
3     state : {red, yellow, green};
4     time : 0..TM;
5     demand : boolean;
6     timeyellow : 0..TYellow;
7
8 ASSIGN
9 init(state) := case
10     cross = id : green;
11     TRUE: red;

```

```

12 esac;
13
14 init(time) := case
15     cross = id : 0;
16     cross != id : TM;
17 esac;
18
19 init(timeyellow) := 0;
20
21 next(state) := case
22     next(cross) = id: case --Si el semaforo tiene el turno
23         state = green : case --Si actualmente esta en verde
24             next(time) < TGreenMin : green; --Que siga en verde si
aun no ha pasado un tiempo minimo
25             demand & next(time) < TGreenMax : green; --Si paso el
tiempo minimo entonces que siga siempre y cuando no pase de un
tiempo maximo
26             TRUE : yellow; --Si llego al tiempo maximo o no hay
demanda, entonces que pase a amarillo
27         esac;
28         state = yellow : case --Si actualmente esta en amarillo
29             next(timeyellow) < TYellow : yellow; --Que siga en
amarillo mientras no haya superado los 3 segundos
30             TRUE: red; --Una vez superado llegue a los 3 segundos,
entonces que pase a rojo
31         esac;
32         state = red : green; --Si es su turno y esta rojo, entonces
significa que ahora debe cambiar a verde
33     esac;
34     TRUE : red; --Si no es su turno, entonces debe mantenerse en
rojo
35 esac;
36
37 next(time) := case
38     next(cross) = id : case --Si es su turno
39         state = red: 0; --Si esta en rojo, eso significa que hay
que empezar el semaforo
40         time < TM: time + 1; --En otro caso, hay que incrementarlo
41         TRUE : TM; --Cuando llegue a TM, que se mantenga ahi
42     esac;
43     TRUE: TM; --Si no es su turno, entonces dejarlo en el tiempo
maximo
44 esac;
45
46 next(demand) := case
47     next(state) = green | next(state) = yellow : {TRUE, FALSE}; --
Si esta en verde o en amarillo, entonces puede seguir la demanda
o apagarse
48     demand = FALSE : {TRUE, FALSE}; --Si no hay demanda, en
cualquier momento puede llegar autos o seguir sin demanda
49     TRUE : TRUE; --Si el siguiente estado no es verde y hay demanda
, entonces esta no va a desaparecer
50 esac;
51
52 next(timeyellow) := case

```



```

53     state = yellow & timeyellow < TYellow : timeyellow + 1;
54     TRUE : 0;
55 esac;
56
57 MODULE main
58 VAR
59     cross : {ns, oe};
60     semaforo_NS : semaforo(cross, ns, TM, TGreenMin, TYellow,
61                             TGreenMax);
62     semaforo_OE : semaforo(cross, oe, TM, TGreenMin, TYellow,
63                             TGreenMax);
64     t_car_arrival : 0..L;
65
66 ASSIGN
67     init(cross) := {ns, oe};
68     init(t_car_arrival) := 0;
69     next(cross) := case
70         cross = ns & semaforo_NS.state = yellow & next(semaforo_NS.
71 timeyellow) = TYellow : oe; --Si el semaforo esta en amarillo y
72 ya termino su tiempo, ahora le toca empezar al otro semaforo
73         cross = oe & semaforo_OE.state = yellow & next(semaforo_OE.
74 timeyellow) = TYellow : ns; --Si el semaforo esta en amarillo y
75 ya termino su tiempo, ahora le toca empezar al otro semaforo
76     TRUE : cross;
77
78     esac;
79     next(t_car_arrival) := case
80
81         cross = ns : case
82             next(cross) = oe : 0;
83             t_car_arrival = L : L; -- Esto le indica al compilador
84 (NuSMV) que la variable no supera el limite
85             semaforo_OE.demand : t_car_arrival + 1;
86             TRUE: 0;
87         esac;
88
89         cross = oe: case
90             next(cross) = ns : 0;
91             t_car_arrival = L : L; -- Esto le indica al compilador
92 (NuSMV) que la variable no supera el limite
93             semaforo_NS.demand : t_car_arrival + 1;
94             TRUE: 0;
95         esac;
96
97     esac;
98
99 DEFINE
100     TYellow := 3;
101     TGreenMax := 15;
102     TGreenMin := 10;
103     TM := TYellow + TGreenMax; -- El tiempo total del ciclo
104     L := 21;
105
106 -- phi 1: Los semaforos de direcciones cruzadas no pueden estar en
107 green simultaneamente.
108 LTLSPEC NAME ambosSemaforosIguales := G(semaforo_NS.state !=

```

```

    semaforo_OE.state); -- Verifica que ambos semaforos no sean
    rojos
99
100 -- phi 2: El yellow debe durar exactamente TYellow y no debe
    saltarse.
101 -- LTLSPEC NAME spec2 := G((semaforo_NS.time >= TGreenMin &
    semaforo_NS.state = yellow) -> X(semaforo_NS.time <= TM &
    semaforo_NS.time >= (TYellow + TGreenMin))) | G((semaforo_NS.
    time >= TGreenMin & semaforo_NS.state = yellow) -> X(semaforo_NS
    .time <= TM & semaforo_NS.time >= (TYellow + TGreenMax)))
102 LTLSPEC NAME spec2 := G(F(semaforo_NS.timeyellow = TYellow) & F(
    semaforo_OE.timeyellow = TYellow))
103
104 -- phi 3: Si hay vehiculos esperando en una direccion y el semaforo
    esta rojo, eventualmente se pondra verde.
105 LTLSPEC NAME demandaYRojo := G((semaforo_NS.demand & semaforo_NS.
    state = red -> F (semaforo_NS.state = green)) & (semaforo_OE.
    demand & semaforo_OE.state = red -> F (semaforo_OE.state = green
    )));
106
107 -- phi 4: El tiempo de verde no debe exceder un maximo Tmax (ej. 30
    segundos).
108 LTLSPEC NAME noExcede := G(!(semaforo_NS.state = green &
    semaforo_NS.time = TGreenMax) & !(semaforo_OE.state = green &
    semaforo_OE.time = TGreenMax));
109
110 -- phi 5: Siempre es posible que algun semaforo este en green
111 CTLSPEC NAME posibleVerde := AG( EF (semaforo_NS.state = green |
    semaforo_OE.state = green));
112
113 -- El tiempo de espera nunca debe exceder 20 ticks
114 CTLSPEC NAME tiempoEspMenor20 := AG (t_car_arrival <= 20);

```

Listing 2: Modelo con tiempo de espera

```

ProyectoSTR-Semaforo main ? * nusmv -int Ejer3.smv
*** This is NuSMV 2.7.1 (compiled on (unknown))
*** Enabled addons are: compass
*** For more information on NuSMV see <http://nusmv.fbk.eu>
*** or email to <nusmv-users@list.fbk.eu>.
*** Please report bugs to <Please report bugs to <nusmv-users@fbk.eu>>

*** Copyright (c) 2010-2026, Fondazione Bruno Kessler

*** This version of NuSMV is linked to the CUDD library version 2.4.1
*** Copyright (c) 1995-2004, Regents of the University of Colorado

NuSMV > go
NuSMV > check_ltlspec
-- specification G semaforo_NS.state != semaforo_OE.state is true
-- specification G ( F semaforo_NS.timeyellow = TYellow & F semaforo_OE.timeyellow = TYellow) is true
-- specification G (((semaforo_NS.demand & semaforo_NS.state = red) -> F semaforo_NS.state = green) & ((semaforo_OE.demand & semaforo_OE.state = red) -> F semaforo_OE.state = green)) is true
-- specification G (!(semaforo_NS.state = green & semaforo_NS.time = TGreenMax) & !(semaforo_OE.state = green & semaforo_OE.time = TGreenMax)) is true
NuSMV > check_ctlspec
-- specification AG (EF (semaforo_NS.state = green | semaforo_OE.state = green)) is true
-- specification AG t_car_arrival <= 20 is true

```

Figura 3: Resultados de verificación para el Ejercicio 3.

Ejercicio 4

Consigna: Incorporar al modelo los botones peatonales. Si el botón peatonal es presionado, lo cual puede ocurrir en cualquier momento, el semáforo debe dar verde peatonal (simulado como una fase especial donde ambos semáforos están en rojo) dentro de los siguientes 10 ticks.

Para el semáforo peatonal:

- Al presionarse el botón para dar cruce peatonal, instantáneamente el semáforo de calle que tenga el cruce (semáforo en *state = green*) realiza el cambio de estado *green* → *yellow*, respetando la cantidad de ticks en la que el mismo debe permanecer en *yellow*, para luego realizar el cambio de estado *yellow* → *red*. A partir de este momento ambos semáforos de calle permanecen en *red* durante 10 ticks, que es la cantidad de tiempo que el semáforo peatonal permite el cruce. Luego de que finalice el tiempo de cruce del semáforo peatonal, el cruce se le devuelve al semáforo que estaba en *green* previo a que se presione el botón de cruce peatonal.

```
1 MODULE semaforo(cross, id, TM, TGreenMin, TYellow, TGreenMax,
  walker_press)
2 VAR
3   state : {red, yellow, green};
4   time : 0..TM;
5   demand : boolean;
6   timeyellow : 0..TYellow;
7
8 ASSIGN
9   init(state) := case
10     cross = id : green;
11     TRUE: red;
12   esac;
13
14   init(time) := case
15     cross = id : 0;
16     cross != id : TM;
17   esac;
18
19   init(timeyellow) := 0;
20
21   next(state) := case
22     state = green & walker_press : yellow; --Si esta en verde y se
    aprieta el boton peatonan entonces que pase automaticamente
    amarillo
23     state = red & walker_press : red; --Si esta en rojo y se apreto
    el boton entonces que se mantenga el semaforo en rojo
24     next(cross) = id: case --Si el semaforo tiene el turno
25       state = green : case --Si actualmente esta en verde
26         next(time) < TGreenMin : green; --Que siga en verde si
    aun no ha pasado un tiempo minimo
27         demand & next(time) < TGreenMax : green; --Si paso el
    tiempo minimo entonces que siga siempre y cuando no pase de un
    tiempo maximo
```

```

28         TRUE : yellow; --Si llego al tiempo maximo o no hay
demanda, entonces que pase a amarillo
29     esac;
30     state = yellow : case --Si actualmente esta en amarillo
31         next(timeyellow) < TYellow : yellow; --Que siga en
amarillo mientras no haya superado los 3 segundos
32         TRUE: red; --Una vez superado llegue a los 3 segundos,
entonces que pase a rojo
33     esac;
34     state = red : green; --Si es su turno y esta rojo, entonces
significa que ahora debe cambiar a verde
35     esac;
36     TRUE : red; --Si no es su turno, entonces debe mantenerse en
rojo
37 esac;
38
39 next(time) := case
40     next(cross) = id : case --Si es su turno
41         state = red: 0; --Si esta en rojo, eso significa que hay
que empezar el semaforo
42         time < TM: time + 1; --En otro caso, hay que incrementarlo
43         TRUE : TM; --Cuando llegue a TM, que se mantenga ahi
44     esac;
45     TRUE: TM; --Si no es su turno, entonces dejarlo en el tiempo
maximo
46 esac;
47
48 next(demand) := case
49     next(state) = green | next(state) = yellow : {TRUE, FALSE}; --
Si esta en verde o en amarillo, entonces puede seguir la demanda
o apagarse
50     demand = FALSE : {TRUE, FALSE}; --Si no hay demanda, en
cualquier momento puede llegar autos o seguir sin demanda
51     TRUE : TRUE; --Si el siguiente estado no es verde y hay demanda
, entonces esta no va a desaparecer
52 esac;
53
54 next(timeyellow) := case
55     state = yellow & timeyellow < TYellow : timeyellow + 1;
56     TRUE : 0;
57 esac;
58
59 MODULE main
60 VAR
61     cross : {ns, oe}; --Variable que indica de que semaforo es el
turno
62     semaforo_NS : semaforo(cross, ns, TM, TGreenMin, TYellow,
TGreenMax, walker_press); --Modulo del semaforo NS
63     semaforo_OE : semaforo(cross, oe, TM, TGreenMin, TYellow,
TGreenMax, walker_press); --Modulo del semaforo OE
64     t_car_arrival : 0..L; --Tiempo maximo que lleva esperando un
auto
65     t_walker_cross : 0..P; --Tiempo que llevan los dos semaforos en
rojo
66     walker_press : boolean; --Variable que controla si se apreto el

```

```

        boton peatonal
67
68 ASSIGN
69     init(cross) := {ns, oe};
70     init(t_car_arrival) := 0;
71     init(t_walker_cross) := 0;
72     init(walker_press) := FALSE;
73
74     next(cross) := case
75         cross = ns & semaforo_NS.state = yellow & next(semaforo_NS.
timeyellow) = TYellow : oe; --Si el semaforo esta en amarillo y
ya termino su tiempo, ahora le toca empezar al otro semaforo
76         cross = oe & semaforo_OE.state = yellow & next(semaforo_OE.
timeyellow) = TYellow : ns; --Si el semaforo esta en amarillo y
ya termino su tiempo, ahora le toca empezar al otro semaforo
77         TRUE : cross;
78     esac;
79     next(t_car_arrival) := case
80
81         cross = ns : case
82             next(cross) = oe : 0;
83             t_car_arrival = L : L; -- Restriccion impuesta para que
el sistema (NuSMV) entienda que la variable no supera L
84             semaforo_OE.demand : t_car_arrival + 1;
85             TRUE: 0;
86         esac;
87
88         cross = oe: case
89             next(cross) = ns : 0;
90             t_car_arrival = L : L; -- Restriccion impuesta para que
el sistema (NuSMV) entienda que la variable no supera L
91             semaforo_NS.demand : t_car_arrival + 1;
92             TRUE: 0;
93         esac;
94     esac;
95
96     next(walker_press) := case
97         !walker_press : {TRUE, FALSE}; -- Si el boton no esta
presionado, puede pasar a estar presionado o no en el siguiente
estado
98         next(t_walker_cross) = P: FALSE; -- Si pasaron los 10 seg
en el que ambos semaforos estaban en rojo, entonces se le quita
el paso al peaton
99         TRUE: TRUE;
100     esac;
101
102     next(t_walker_cross) := case
103         t_walker_cross = P : P; -- Restriccion impuesta para que el
sistema (NuSMV) entienda que la variable no supera P
104         walker_press & semaforo_NS.state = semaforo_OE.state :
t_walker_cross + 1;
105         TRUE : 0;
106     esac;
107
108 DEFINE

```

```

109     TYellow := 3; --Tiempo que estara en amarillo el semaforo
110     TGreenMax := 15; -- Tiempo maximo del semaforo verde si hay
demanda
111     TGreenMin := 10; -- Tiempo minimo del semaforo verde con o sin
demanda
112     TM := TYellow + TGreenMax; -- El tiempo total del ciclo
113     L := 21; -- Tiempo que no debe exceder un auto en espera
114     P := 10; -- Tiempo que tienen los peatones para pasar
115
116
117 -- phi 1: Los semaforos de direcciones cruzadas no pueden estar en
green simultaneamente.
118 LTLSPEC NAME ambosSemaforosIguales := !G(semaforo_NS.state = green
& semaforo_OE.state = green);
119
120 -- phi 2: El yellow debe durar exactamente TYellow y no debe
saltarse.
121 -- LTLSPEC NAME spec2 := G((semaforo_NS.time >= TGreenMin &
semaforo_NS.state = yellow) -> X(semaforo_NS.time <= TM &
semaforo_NS.time >= (TYellow + TGreenMin))) | G((semaforo_NS.
time >= TGreenMin & semaforo_NS.state = yellow) -> X(semaforo_NS
.time <= TM & semaforo_NS.time >= (TYellow + TGreenMax)))
122 LTLSPEC NAME spec2 := G(F(semaforo_NS.timeyellow = TYellow) & F(
semaforo_OE.timeyellow = TYellow))
123
124 -- phi 3: Si hay vehiculos esperando en una direccion y el semaforo
esta rojo, eventualmente se pondra verde.
125 LTLSPEC NAME demandaYRojo := G((semaforo_NS.demand & semaforo_NS.
state = red -> F (semaforo_NS.state = green)) & (semaforo_OE.
demand & semaforo_OE.state = red -> F (semaforo_OE.state = green
)));
126
127 -- phi 4: El tiempo de verde no debe exceder un maximo Tmax (ej. 30
segundos).
128 LTLSPEC NAME noExcede := G(!(semaforo_NS.state = green &
semaforo_NS.time = TGreenMax) & !(semaforo_OE.state = green &
semaforo_OE.time = TGreenMax));
129
130 -- phi 5: Siempre es posible que algun semaforo este en green
131 CTLSPEC NAME posibleVerde := AG( EF (semaforo_NS.state = green |
semaforo_OE.state = green));
132
133 -- El tiempo de espera nunca debe exceder 20 ticks
134 CTLSPEC NAME tiempoEspMenor20 := AG (t_car_arrival <= 20);

```

Listing 3: Modelo con botón peatonal

```

ProyectoSTR-Semaforo docs/report } nusmv -int Ejer4.smv
*** This is NuSMV 2.7.1 (compiled on (unknown))
*** Enabled addons are: compass
*** For more information on NuSMV see <http://nusmv.fbk.eu>
*** or email to <nusmv-users@list.fbk.eu>.
*** Please report bugs to <Please report bugs to <nusmv-users@fbk.eu>>

*** Copyright (c) 2010-2026, Fondazione Bruno Kessler

*** This version of NuSMV is linked to the CUDD library version 2.4.1
*** Copyright (c) 1995-2004, Regents of the University of Colorado

NuSMV > go
NuSMV > check_ltlspec
-- specification !( G (semaforo_NS.state = green & semaforo_OE.state = green)) is true
-- specification G ( F semaforo_NS.timeyellow = TYellow & F semaforo_OE.timeyellow = TYellow) is true
-- specification G (((semaforo_NS.demand & semaforo_NS.state = red) -> F semaforo_NS.state = green) & (
(semaforo_OE.demand & semaforo_OE.state = red) -> F semaforo_OE.state = green)) is true
-- specification G (!(semaforo_NS.state = green & semaforo_NS.time = TGreenMax) & !(semaforo_OE.state =
green & semaforo_OE.time = TGreenMax)) is true
NuSMV > check_ctlspec
-- specification AG (EF (semaforo_NS.state = green | semaforo_OE.state = green)) is true
-- specification AG t_car_arrival <= 20 is false
-- as demonstrated by the following execution sequence
Trace Description: CTL Counterexample
Trace Type: Counterexample
-> State: 1.1 <-
  cross = oe
  semaforo_NS.state = red
  semaforo_NS.time = 18
  semaforo_NS.demand = FALSE
  semaforo_NS.timeyellow = 0
  semaforo_OE.state = green
  semaforo_OE.time = 0
  semaforo_OE.demand = FALSE
  semaforo_OE.timeyellow = 0
  t_car_arrival = 0
  t_walker_cross = 0
  walker_press = FALSE
  P = 10
  L = 21
  TM = 18
  TGreenMin = 10
  TGreenMax = 15
  TYellow = 3
-> State: 1.2 <-
  semaforo_OE.time = 1
  walker_press = TRUE
-> State: 1.3 <-
  semaforo_OE.state = yellow
  semaforo_OE.time = 2
-> State: 1.4 <-
  semaforo_OE.time = 3
  semaforo_OE.timeyellow = 1
-> State: 1.5 <-
  semaforo_OE.time = 4
  semaforo_OE.timeyellow = 2

```

Figura 4: Resultados de verificación para el Ejercicio 4 (Parte 1).

```

-> State: 1.7 <-
  semaforo_0E.timeyellow = 0
  t_car_arrival = 1
  t_walker_cross = 1
-> State: 1.8 <-
  t_car_arrival = 2
  t_walker_cross = 2
-> State: 1.9 <-
  t_car_arrival = 3
  t_walker_cross = 3
-> State: 1.10 <-
  t_car_arrival = 4
  t_walker_cross = 4
-> State: 1.11 <-
  t_car_arrival = 5
  t_walker_cross = 5
-> State: 1.12 <-
  t_car_arrival = 6
  t_walker_cross = 6
-> State: 1.13 <-
  t_car_arrival = 7
  t_walker_cross = 7
-> State: 1.14 <-
  t_car_arrival = 8
  t_walker_cross = 8
-> State: 1.15 <-
  t_car_arrival = 9
  t_walker_cross = 9
-> State: 1.16 <-
  t_car_arrival = 10
  t_walker_cross = 10
  walker_press = FALSE
-> State: 1.17 <-
  semaforo_NS.state = green
  t_car_arrival = 11
-> State: 1.18 <-
  semaforo_NS.time = 1
  t_car_arrival = 12
-> State: 1.19 <-
  semaforo_NS.time = 2
  t_car_arrival = 13
-> State: 1.20 <-
  semaforo_NS.time = 3
  t_car_arrival = 14
-> State: 1.21 <-
  semaforo_NS.time = 4
  t_car_arrival = 15
-> State: 1.22 <-
  semaforo_NS.time = 5
  t_car_arrival = 16
-> State: 1.23 <-
  semaforo_NS.time = 6
  t_car_arrival = 17
-> State: 1.24 <-
  semaforo_NS.time = 7
  t_car_arrival = 18

```

Figura 5: Resultados de verificación para el Ejercicio 4 (Parte 2).


```

-> State: 1.25 <-
  semaforo_NS.time = 8
  t_car_arrival = 19
  walker_press = TRUE
-> State: 1.26 <-
  semaforo_NS.state = yellow
  semaforo_NS.time = 9
  t_car_arrival = 20
  walker_press = FALSE
-> State: 1.27 <-
  semaforo_NS.time = 10
  semaforo_NS.timeyellow = 1
  t_car_arrival = 21

```

Figura 6: Resultados de verificación para el Ejercicio 4 (Parte 3).

Ejercicio 5

Consigna: Modificar el modelo para introducir errores deliberados que violen cada una de las propiedades establecidas. Analizar el contraejemplo generado por NuSMV.

Consideraciones que tuvimos para realizar los contraejemplos:

- Usar un error para violar múltiples propiedades.
- Insertar múltiples errores que rompan múltiples propiedades en un mismo código en vez de crear modelos separados para romper las propiedades de manera individual. Igualmente, debido a la imposibilidad de violar la propiedad 1 y 5 al mismo tiempo, se realizaron 2 modelos con errores.

```

1  -- Para violar las propiedades 1, 2 y 3
2  MODULE semaforo(cross, id, TM, TGreenMin, TYellow, TGreenMax,
   walker_press)
3  VAR
4      state : {red, yellow, green};
5      time : 0..TM;
6      demand : boolean;
7      timeyellow : 0..TYellow;
8
9  ASSIGN
10 init(state) := case
11     cross = id : green;
12     TRUE: red;
13 esac;
14
15 init(time) := case
16     cross = id : 0;
17     cross != id : TM;
18 esac;
19
20 init(timeyellow) := 0;
21
22 next(state) := case
23     state = green & walker_press : yellow; --Si esta en verde y se
   aprieta el boton peatonan entonces que pase automaticamente
   amarillo

```

```

24     state = red & walker_press : red; --Si esta en rojo y se apreto
    el boton entonces que se mantenga el semaforo en rojo
25     next(cross) = id: case --Si el semaforo tiene el turno
26         state = green : case --Si actualmente esta en verde
27             next(time) < TGreenMin : green; --Que siga en verde si
    aun no ha pasado un tiempo minimo
28             demand & next(time) < TGreenMax : green; --Si paso el
    tiempo minimo entonces que siga siempre y cuando no pase de un
    tiempo maximo
29             TRUE : yellow; --Si llego al tiempo maximo o no hay
    demanda, entonces que pase a amarillo
30         esac;
31         state = yellow : case --Si actualmente esta en amarillo
32             next(timeyellow) < TYellow : yellow; --Que siga en
    amarillo mientras no haya superado los 3 segundos
33             TRUE: red; --Una vez superado llegue a los 3 segundos,
    entonces que pase a rojo
34         esac;
35         state = red : green; --Si es su turno y esta rojo, entonces
    significa que ahora debe cambiar a verde
36     esac;
37     TRUE : red; --Si no es su turno, entonces debe mantenerse en
    rojo
38 esac;
39
40 next(time) := case
41     next(cross) = id : case --Si es su turno
42         state = red: 0; --Si esta en rojo, eso significa que hay
    que empezar el semaforo
43         time < TM: time + 1; --En otro caso, hay que incrementarlo
44         TRUE : TM; --Cuando llegue a TM, que se mantenga ahi
45     esac;
46     TRUE: TM; --Si no es su turno, entonces dejarlo en el tiempo
    maximo
47 esac;
48
49 next(demand) := case
50     next(state) = green | next(state) = yellow : {TRUE, FALSE}; --
    Si esta en verde o en amarillo, entonces puede seguir la demanda
    o apagarse
51     demand = FALSE : {TRUE, FALSE}; --Si no hay demanda, en
    cualquier momento puede llegar autos o seguir sin demanda
52     TRUE : TRUE; --Si el siguiente estado no es verde y hay demanda
    , entonces esta no va a desaparecer
53 esac;
54
55 next(timeyellow) := case
56     state = yellow & timeyellow < TYellow : timeyellow + 1;
57     TRUE : 0;
58 esac;
59
60 MODULE main
61 VAR
62     cross : {ns, oe}; --Variable que indica de que semaforo es el
    turno

```

```

63     semaforo_NS : semaforo(cross, ns, TM, TGreenMin, TYellow,
64     TGreenMax, walker_press); --Modulo del semaforo NS
65     semaforo_OE : semaforo(cross, oe, TM, TGreenMin, TYellow,
66     TGreenMax, walker_press); --Modulo del semaforo OE
67     t_car_arrival : 0..L; --Tiempo maximo que lleva esperando un
68     auto
69     t_walker_cross : 0..P; --Tiempo que llevan los dos semaforos en
70     rojo
71     walker_press : boolean; --Variable que controla si se apreto el
72     boton peatonal
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102

```

ASSIGN

```

    init(cross) := {ns, oe};
    init(t_car_arrival) := 0;
    init(t_walker_cross) := 0;
    init(walker_press) := FALSE;

    -- Aqui se introdujo error, modificando el proximo estado para
    el cruce, haciendo que den todas las especificaciones faltas,
    excepto \phi_5
    next(cross) := case
        cross = ns: oe; --CAMBIO: Siempre estan alternando los
        semaforos
        cross = oe: ns; --CAMBIO: Siempre estan alternando los
        semaforos
        TRUE : cross;
    esac;
    next(t_car_arrival) := case
        cross = ns : case
            next(cross) = oe : 0;
            t_car_arrival = L : L; -- Restriccion impuesta para que
            el sistema (NuSMV) entienda que la variable no supera L
            semaforo_OE.demand : t_car_arrival + 1;
            TRUE: 0;
        esac;
        cross = oe: case
            next(cross) = ns : 0;
            t_car_arrival = L : L; -- Restriccion impuesta para que
            el sistema (NuSMV) entienda que la variable no supera L
            semaforo_NS.demand : t_car_arrival + 1;
            TRUE: 0;
        esac;
    esac;

    next(walker_press) := case
        !walker_press : {TRUE, FALSE}; -- Si el boton no esta
        presionado, puede pasar a estar presionado o no en el siguiente
        estado
        next(t_walker_cross) = P: FALSE; -- Si pasaron los 10 seg
        en el que ambos semaforos estaban en rojo, entonces se le quita
        el paso al peaton
        TRUE: TRUE;
    esac;

```

```

103
104     next(t_walker_cross) := case
105         t_walker_cross = P : P; -- Restriccion impuesta para que el
106             sistema (NuSMV) entienda que la variable no supera P
107         walker_press & semaforo_NS.state = semaforo_OE.state :
108             t_walker_cross + 1;
109         TRUE : 0;
110     esac;
111
112 DEFINE
113     TYellow := 3; --Tiempo que estara en amarillo el semaforo
114     TGreenMax := 15; -- Tiempo maximo del semaforo verde si hay
115     demanda
116     TGreenMin := 10; -- Tiempo minimo del semaforo verde con o sin
117     demanda
118     TM := TYellow + TGreenMax; -- El tiempo total del ciclo
119     L := 21; -- Tiempo que no debe exceder un auto en espera
120     P := 10; -- Tiempo que tienen los peatones para pasar
121
122 -- phi 1: Los semaforos de direcciones cruzadas no pueden estar en
123 green simultaneamente.
124 LTLSPEC NAME ambosSemaforosIguales := G(semaforo_NS.state = green &
125     semaforo_OE.state = green);
126
127 -- phi 2: El yellow debe durar exactamente TYellow y no debe
128 saltarse.
129 LTLSPEC NAME spec2 := G((semaforo_NS.time >= TGreenMin &
130     semaforo_NS.state = yellow) -> X(semaforo_NS.time <= TM &
131     semaforo_NS.time >= (TYellow + TGreenMin))) | G((semaforo_NS.
132     time >= TGreenMin & semaforo_NS.state = yellow) -> X(semaforo_NS.
133     .time <= TM & semaforo_NS.time >= (TYellow + TGreenMax)))
134 LTLSPEC NAME spec2 := G(F(semaforo_NS.timeyellow = TYellow) & F(
135     semaforo_OE.timeyellow = TYellow))
136
137 -- phi 3: Si hay vehiculos esperando en una direccion y el semaforo
138 esta rojo, eventualmente se pondra verde.
139 LTLSPEC NAME demandaYRojo := G((semaforo_NS.demand & semaforo_NS.
140     state = red -> F (semaforo_NS.state = green)) & (semaforo_OE.
141     demand & semaforo_OE.state = red -> F (semaforo_OE.state = green
142     )));
143
144 -- phi 4: El tiempo de verde no debe exceder un maximo Tmax (ej. 30
145 segundos).
146 LTLSPEC NAME noExcede := G(!(semaforo_NS.state = green &
147     semaforo_NS.time = TGreenMax) & !(semaforo_OE.state = green &
148     semaforo_OE.time = TGreenMax));
149
150 -- phi 5: Siempre es posible que algun semaforo este en green
151 CTLSPEC NAME posibleVerde := AG( EF (semaforo_NS.state = green |
152     semaforo_OE.state = green));
153
154 -- El tiempo de espera nunca debe exceder 20 ticks
155 CTLSPEC NAME tiempoEspMenor20 := AG (t_car_arrival <= 20);
156
157

```

Listing 4: Modelo con errores introducidos para violar specs 1, 2 y 3

```

1  -- Para violar las propiedades 4 y 5
2  MODULE semaforo(cross, id, TM, TGreenMin, TYellow, TGreenMax,
   walker_press)
3  VAR
4      state : {red, yellow, green};
5      time : 0..TM;
6      demand : boolean;
7      timeyellow : 0..TYellow;
8
9  ASSIGN
10 init(state) := case
11     cross = id : green;
12     TRUE: red;
13 esac;
14
15 init(time) := case
16     cross = id : 0;
17     cross != id : TM;
18 esac;
19
20 init(timeyellow) := 0;
21
22 next(state) := case
23     state = green & walker_press : yellow; --Si esta en verde y se
   aprieta el boton peatonan entonces que pase automaticamente
   amarillo
24     state = red & walker_press : red; --Si esta en rojo y se apreto
   el boton entonces que se mantenga el semaforo en rojo
25     next(cross) = id: case --Si el semaforo tiene el turno
26         state = green : case --Si actualmente esta en verde
27             next(time) < TGreenMin : green; --CAMBIO: Que siga en
   verde si aun no ha pasado un tiempo minimo
28             demand & next(time) < TM+1: green; --Ahora esta en
   verde mas de TM ticks
29             TRUE : yellow; --Si llego al tiempo maximo o no hay
   demanda, entonces que pase a amarillo
30         esac;
31         state = yellow : case --Si actualmente esta en amarillo
32             next(timeyellow) < TYellow : yellow; --Que siga en
   amarillo mientras no haya superado los 3 segundos
33             TRUE: red; --Una vez superado llegue a los 3 segundos,
   entonces que pase a rojo
34         esac;
35         state = red : red; --CAMBIO: Si esta en rojo nunca cambiara
   a verde
36     esac;
37     TRUE : red; --Si no es su turno, entonces debe mantenerse en
   rojo
38 esac;
39
40 next(time) := case
41     next(cross) = id : case --Si es su turno

```

```

42     state = red: 0; --Si esta en rojo, eso significa que hay
    que empezar el semaforo
43     time < TM: time + 1; --En otro caso, hay que incrementarlo
44     TRUE : TM; --Cuando llegue a TM, que se mantenga ahi
45     esac;
46     TRUE: TM; --Si no es su turno, entonces dejarlo en el tiempo
    maximo
47 esac;
48
49 next(demand) := case
50     next(state) = green | next(state) = yellow : {TRUE, FALSE}; --
    Si esta en verde o en amarillo, entonces puede seguir la demanda
    o apagarse
51     demand = FALSE : {TRUE, FALSE}; --Si no hay demanda, en
    cualquier momento puede llegar autos o seguir sin demanda
52     TRUE : TRUE; --Si el siguiente estado no es verde y hay demanda
    , entonces esta no va a desaparecer
53 esac;
54
55 next(timeyellow) := case
56     state = yellow & timeyellow < TYellow : timeyellow + 1;
57     TRUE : 0;
58 esac;
59
60 MODULE main
61 VAR
62     cross : {ns, oe}; --Variable que indica de que semaforo es el
    turno
63     semaforo_NS : semaforo(cross, ns, TM, TGreenMin, TYellow,
    TGreenMax, walker_press); --Modulo del semaforo NS
64     semaforo_OE : semaforo(cross, oe, TM, TGreenMin, TYellow,
    TGreenMax, walker_press); --Modulo del semaforo OE
65     t_car_arrival : 0..L; --Tiempo maximo que lleva esperando un
    auto
66     t_walker_cross : 0..P; --Tiempo que llevan los dos semaforos en
    rojo
67     walker_press : boolean; --Variable que controla si se apreto el
    boton peatonal
68
69 ASSIGN
70     init(cross) := {ns, oe};
71     init(t_car_arrival) := 0;
72     init(t_walker_cross) := 0;
73     init(walker_press) := FALSE;
74
75     next(cross) := case
76         cross = ns & semaforo_NS.state = yellow & next(semaforo_NS.
    timeyellow) = TYellow : oe; --Si el semaforo esta en amarillo y
    ya termino su tiempo, ahora le toca empezar al otro semaforo
77         cross = oe & semaforo_OE.state = yellow & next(semaforo_OE.
    timeyellow) = TYellow : ns; --Si el semaforo esta en amarillo y
    ya termino su tiempo, ahora le toca empezar al otro semaforo
78         TRUE : cross;
79     esac;
80     next(t_car_arrival) := case

```

```

81
82     cross = ns : case
83         next(cross) = oe : 0;
84         t_car_arrival = L : L; -- Restriccion impuesta para que
85         el sistema (NuSMV) entienda que la variable no supera L
86         semaforo_OE.demand : t_car_arrival + 1;
87         TRUE: 0;
88     esac;
89
90     cross = oe: case
91         next(cross) = ns : 0;
92         t_car_arrival = L : L; -- Restriccion impuesta para que
93         el sistema (NuSMV) entienda que la variable no supera L
94         semaforo_NS.demand : t_car_arrival + 1;
95         TRUE: 0;
96     esac;
97
98     next(walker_press) := case
99         !walker_press : {TRUE, FALSE}; -- Si el boton no esta
100         presionado, puede pasar a estar presionado o no en el siguiente
101         estado
102         next(t_walker_cross) = P: FALSE; -- Si pasaron los 10 seg
103         en el que ambos semaforos estaban en rojo, entonces se le quita
104         el paso al peaton
105         TRUE: TRUE;
106     esac;
107
108     next(t_walker_cross) := case
109         t_walker_cross = P : P; -- Restriccion impuesta para que el
110         sistema (NuSMV) entienda que la variable no supera P
111         walker_press & semaforo_NS.state = semaforo_OE.state :
112         t_walker_cross + 1;
113         TRUE : 0;
114     esac;
115
116 DEFINE
117     TYellow := 3; --Tiempo que estara en amarillo el semaforo
118     TGreenMax := 15; -- Tiempo maximo del semaforo verde si hay
119     demanda
120     TGreenMin := 10; -- Tiempo minimo del semaforo verde con o sin
121     demanda
122     TM := TYellow + TGreenMax; -- El tiempo total del ciclo
123     L := 21; -- Tiempo que no debe exceder un auto en espera
124     P := 10; -- Tiempo que tienen los peatones para pasar
125
126
127 -- phi 1: Los semaforos de direcciones cruzadas no pueden estar en
128 green simultaneamente.
129 LTLSPEC NAME ambosSemaforosIguales := G(semaforo_NS.state = green &
130     semaforo_OE.state = green);
131
132
133 -- phi 2: El yellow debe durar exactamente TYellow y no debe
134 saltarse.
135 LTLSPEC NAME spec2 := G((semaforo_NS.time >= TGreenMin &

```

```

    semaforo_NS.state = yellow) -> X(semaforo_NS.time <= TM &
    semaforo_NS.time >= (TYellow + TGreenMin))) | G((semaforo_NS.
    time >= TGreenMin & semaforo_NS.state = yellow) -> X(semaforo_NS
    .time <= TM & semaforo_NS.time >= (TYellow + TGreenMax)))
123 LTLSPEC NAME spec2 := G(F(semaforo_NS.timeyellow = TYellow) & F(
    semaforo_OE.timeyellow = TYellow))
124
125 -- phi 3: Si hay vehiculos esperando en una direccion y el semaforo
    esta rojo, eventualmente se pondra verde.
126 LTLSPEC NAME demandaYRojo := G((semaforo_NS.demand & semaforo_NS.
    state = red -> F (semaforo_NS.state = green)) & (semaforo_OE.
    demand & semaforo_OE.state = red -> F (semaforo_OE.state = green
    )));
127
128 -- phi 4: El tiempo de verde no debe exceder un maximo Tmax (ej. 30
    segundos).
129 LTLSPEC NAME noExcede := G(!(semaforo_NS.state = green &
    semaforo_NS.time = TGreenMax) & !(semaforo_OE.state = green &
    semaforo_OE.time = TGreenMax));
130
131 -- phi 5: Siempre es posible que algun semaforo este en green
132 CTLSPEC NAME posibleVerde := AG( EF (semaforo_NS.state = green |
    semaforo_OE.state = green));
133
134 -- El tiempo de espera nunca debe exceder 20 ticks
135 CTLSPEC NAME tiempoEspMenor20 := AG (t_car_arrival <= 20);

```

Listing 5: Modelo con errores introducidos para violar specs 4 y 5


```

ProyectoSTR-Semaforo main • ? ) nusmv -int Ejer5_spec1_2_3.smv
*** This is NuSMV 2.7.1 (compiled on (unknown))
*** Enabled addons are: compass
*** For more information on NuSMV see <http://nusmv.fbk.eu>
*** or email to <nusmv-users@list.fbk.eu>.
*** Please report bugs to <Please report bugs to <nusmv-users@fbk.eu>>

*** Copyright (c) 2010-2026, Fondazione Bruno Kessler

*** This version of NuSMV is linked to the CUDD library version 2.4.1
*** Copyright (c) 1995-2004, Regents of the University of Colorado

NuSMV > go
NuSMV > check_ltlspec
-- specification  G (semaforo_NS.state = green & semaforo_OE.state = green)  is false
-- as demonstrated by the following execution sequence
Trace Description: LTL Counterexample
Trace Type: Counterexample
-- Loop starts here
-> State: 1.1 <-
  cross = oe
  semaforo_NS.state = red
  semaforo_NS.time = 18
  semaforo_NS.demand = FALSE
  semaforo_NS.timeyellow = 0
  semaforo_OE.state = green
  semaforo_OE.time = 0
  semaforo_OE.demand = FALSE
  semaforo_OE.timeyellow = 0
  t_car_arrival = 0
  t_walker_cross = 0
  walker_press = FALSE
  P = 10
  L = 21
  TM = 18
  TGreenMin = 10
  TGreenMax = 15
  TYellow = 3
-> State: 1.2 <-
  cross = ns
  semaforo_NS.state = green
  semaforo_NS.time = 0
  semaforo_OE.state = red
  semaforo_OE.time = 18
-> State: 1.3 <-
  cross = oe
  semaforo_NS.state = red
  semaforo_NS.time = 18
  semaforo_OE.state = green
  semaforo_OE.time = 0

```

Figura 7: Contraejemplo para la especificación 1.

```

-- specification G ( F semaforo_NS.timeyellow = TYellow & F semaforo_OE.timeyellow = TYellow) is false
-- as demonstrated by the following execution sequence
Trace Description: LTL Counterexample
Trace Type: Counterexample
-- Loop starts here
-> State: 2.1 <-
  cross = oe
  semaforo_NS.state = red
  semaforo_NS.time = 18
  semaforo_NS.demand = FALSE
  semaforo_NS.timeyellow = 0
  semaforo_OE.state = green
  semaforo_OE.time = 0
  semaforo_OE.demand = FALSE
  semaforo_OE.timeyellow = 0
  t_car_arrival = 0
  t_walker_cross = 0
  walker_press = FALSE
  P = 10
  L = 21
  TM = 18
  TGreenMin = 10
  TGreenMax = 15
  TYellow = 3
-> State: 2.2 <-
  cross = ns
  semaforo_NS.state = green
  semaforo_NS.time = 0
  semaforo_OE.state = red
  semaforo_OE.time = 18
-> State: 2.3 <-
  cross = oe
  semaforo_NS.state = red
  semaforo_NS.time = 18
  semaforo_OE.state = green
  semaforo_OE.time = 0

```

Figura 8: Contraejemplo para la especificación 2.

```

-- specification  G (((semaforo_NS.demand & semaforo_NS.state = red) ->  F s
semaforo_NS.state = green) & ((semaforo_OE.demand & semaforo_OE.state = red)
->  F semaforo_OE.state = green))  is false
-- as demonstrated by the following execution sequence
Trace Description: LTL Counterexample
Trace Type: Counterexample
-> State: 3.1 <-
  cross = oe
  semaforo_NS.state = red
  semaforo_NS.time = 18
  semaforo_NS.demand = FALSE
  semaforo_NS.timeyellow = 0
  semaforo_OE.state = green
  semaforo_OE.time = 0
  semaforo_OE.demand = FALSE
  semaforo_OE.timeyellow = 0
  t_car_arrival = 0
  t_walker_cross = 0
  walker_press = FALSE
  P = 10
  L = 21
  TM = 18
  TGreenMin = 10
  TGreenMax = 15
  TYellow = 3
-> State: 3.2 <-
  cross = ns
  semaforo_NS.state = green
  semaforo_NS.time = 0
  semaforo_OE.state = red
  semaforo_OE.time = 18
  semaforo_OE.demand = TRUE
  walker_press = TRUE
-> State: 3.3 <-
  cross = oe
  semaforo_NS.state = yellow
  semaforo_NS.time = 18
  semaforo_OE.time = 0
-> State: 3.4 <-
  cross = ns
  semaforo_NS.timeyellow = 1
  semaforo_OE.time = 18
-> State: 3.5 <-
  cross = oe
  semaforo_NS.state = red
  semaforo_NS.timeyellow = 2
  semaforo_OE.time = 0
-> State: 3.6 <-
  cross = ns
  semaforo_NS.time = 0
  semaforo_NS.timeyellow = 0
  semaforo_OE.time = 18
  t_walker_cross = 1

```

Figura 9: Contraejemplo para la especificación 3.

```

-> State: 3.7 <-
  cross = oe
  semaforo_NS.time = 18
  semaforo_OE.time = 0
  t_walker_cross = 2
-> State: 3.8 <-
  cross = ns
  semaforo_NS.time = 0
  semaforo_OE.time = 18
  t_walker_cross = 3
-> State: 3.9 <-
  cross = oe
  semaforo_NS.time = 18
  semaforo_OE.time = 0
  t_walker_cross = 4
-> State: 3.10 <-
  cross = ns
  semaforo_NS.time = 0
  semaforo_OE.time = 18
  t_walker_cross = 5
-> State: 3.11 <-
  cross = oe
  semaforo_NS.time = 18
  semaforo_OE.time = 0
  t_walker_cross = 6
-> State: 3.12 <-
  cross = ns
  semaforo_NS.time = 0
  semaforo_OE.time = 18
  t_walker_cross = 7
-> State: 3.13 <-
  cross = oe
  semaforo_NS.time = 18
  semaforo_OE.time = 0
  t_walker_cross = 8
-> State: 3.14 <-
  cross = ns
  semaforo_NS.time = 0
  semaforo_OE.time = 18
  t_walker_cross = 9
-> State: 3.15 <-
  cross = oe
  semaforo_NS.time = 18
  semaforo_OE.time = 0
  t_walker_cross = 10
  walker_press = FALSE
-- Loop starts here
-> State: 3.16 <-
  cross = ns
  semaforo_NS.state = green
  semaforo_NS.time = 0
  semaforo_OE.time = 18
  walker_press = TRUE
-> State: 3.17 <-
  cross = oe
  semaforo_NS.state = yellow
  semaforo_NS.time = 18
  semaforo_OE.time = 0
  walker_press = FALSE

```

Figura 10: Contraejemplo para la especificación 4.

```
-> State: 3.17 <-  
  cross = oe  
  semaforo_NS.state = yellow  
  semaforo_NS.time = 18  
  semaforo_OE.time = 0  
  walker_press = FALSE  
-> State: 3.18 <-  
  cross = ns  
  semaforo_NS.timeyellow = 1  
  semaforo_OE.time = 18  
  walker_press = TRUE  
-> State: 3.19 <-  
  cross = oe  
  semaforo_NS.state = red  
  semaforo_NS.timeyellow = 2  
  semaforo_OE.time = 0  
  walker_press = FALSE  
-> State: 3.20 <-  
  cross = ns  
  semaforo_NS.state = green  
  semaforo_NS.time = 0  
  semaforo_NS.timeyellow = 0  
  semaforo_OE.time = 18  
  walker_press = TRUE
```

Figura 11: Contraejemplo para la especificación 5.

```

-- specification 6 (! (semaforo_NS.state = green & semaforo_NS.time = TGreenMax) & !(semaforo_
OE.state = green & semaforo_OE.time = TGreenMax)) is false
-- as demonstrated by the following execution sequence
Trace Description: LTL Counterexample
Trace Type: Counterexample
-> State: 4.1 <-
  cross = oe
  semaforo_NS.state = red
  semaforo_NS.time = 18
  semaforo_NS.demand = FALSE
  semaforo_NS.timeyellow = 0
  semaforo_OE.state = green
  semaforo_OE.time = 0
  semaforo_OE.demand = FALSE
  semaforo_OE.timeyellow = 0
  t_car_arrival = 0
  t_walker_cross = 0
  walker_press = FALSE
  P = 10
  L = 21
  TM = 18
  TGreenMin = 10
  TGreenMax = 15
  TYellow = 3
-> State: 4.2 <-
  semaforo_OE.time = 1
-> State: 4.3 <-
  semaforo_OE.time = 2
-> State: 4.4 <-
  semaforo_OE.time = 3
-> State: 4.5 <-
  semaforo_OE.time = 4
-> State: 4.6 <-
  semaforo_OE.time = 5
-> State: 4.7 <-
  semaforo_OE.time = 6
-> State: 4.8 <-
  semaforo_OE.time = 7
-> State: 4.9 <-
  semaforo_NS.demand = TRUE
  semaforo_OE.time = 8
-> State: 4.10 <-
  semaforo_OE.time = 9
  semaforo_OE.demand = TRUE
  t_car_arrival = 1
-> State: 4.11 <-
  semaforo_OE.time = 10
  t_car_arrival = 2
-> State: 4.12 <-
  semaforo_OE.time = 11
  t_car_arrival = 3
-> State: 4.13 <-
  semaforo_OE.time = 12
  t_car_arrival = 4
-> State: 4.14 <-
  semaforo_OE.time = 13
  t_car_arrival = 5

```

Figura 12: Contraejemplo para la especificación 6.

```

-> State: 4.15 <-
  semaforo_0E.time = 14
  t_car_arrival = 6
-> State: 4.16 <-
  semaforo_0E.time = 15
  semaforo_0E.demand = FALSE
  t_car_arrival = 7
-> State: 4.17 <-
  semaforo_0E.state = yellow
  semaforo_0E.time = 16
  t_car_arrival = 8
-> State: 4.18 <-
  semaforo_0E.time = 17
  semaforo_0E.timeyellow = 1
  t_car_arrival = 9
-> State: 4.19 <-
  semaforo_0E.time = 18
  semaforo_0E.timeyellow = 2
  t_car_arrival = 10
-> State: 4.20 <-
  cross = ns
  semaforo_NS.time = 0
  semaforo_0E.state = red
  semaforo_0E.timeyellow = 3
  t_car_arrival = 0
-- Loop starts here
-> State: 4.21 <-
  semaforo_0E.timeyellow = 0
-> State: 4.22 <-

```

Figura 13: Contraejemplo para la especificación 7.

```

NuSMV > check_ctlspec
-- specification AG (EF (semaforo_NS.state = green | semaforo_0E.state = green)) is false
-- as demonstrated by the following execution sequence
Trace Description: CTL Counterexample
Trace Type: Counterexample
-> State: 5.1 <-
  cross = oe
  semaforo_NS.state = red
  semaforo_NS.time = 18
  semaforo_NS.demand = FALSE
  semaforo_NS.timeyellow = 0
  semaforo_0E.state = green
  semaforo_0E.time = 0
  semaforo_0E.demand = FALSE
  semaforo_0E.timeyellow = 0
  t_car_arrival = 0
  t_walker_cross = 0
  walker_press = FALSE
  P = 10
  L = 21
  TM = 18
  TGreenMin = 10
  TGreenMax = 15
  TYellow = 3
-> State: 5.2 <-
  semaforo_0E.time = 1
  walker_press = TRUE
-> State: 5.3 <-
  semaforo_NS.demand = TRUE
  semaforo_0E.state = yellow
  semaforo_0E.time = 2

```

Figura 14: Contraejemplo para la especificación 8.

Ejercicio 6

Consigna: Para el modelo final (con demanda y botón peatonal), medir el tiempo de verificación y el número de estados alcanzables usando el comando `print reachable states` en NuSMV. Comparar con el modelo cíclico fijo. Discutir la complejidad del espacio de estados.

Para el modelo inicial:

```
1 #####
2 system diameter: 2
3 reachable states: 4 (2^2) out of 3528 (2^11.7846)
4 #####
```

Listing 6: Resultado de ejecutar el comando `print_reachable_states` para el modelo inicial

El resultado que muestra el comando `print_reachable_states`, indica que hay 4 estados alcanzables y que hay 3528 combinaciones posibles con la especificación del modelo inicial. El `system diameter = 2`, indica que todo estado alcanzable desde el estado inicial puede alcanzarse en a lo sumo 2 transiciones.

Esto es debido a la variable `cross`, ya que se encarga de realizar el cambio entre las direcciones `ns` y `oe`, solo teniendo en cuenta si el semáforo que tiene el paso llega al estado amarillo. Estableciendo que en el próximo estado se le otorga el paso al otro semáforo. Esto genera un ciclo predecible, que genera pocas configuraciones de estados durante la ejecución.

Para el modelo Final:

```
1 #####
2 system diameter: 34
3 reachable states: 13596 (2^13.7309) out of 2.01282e+08 (2^27.5846)
4 #####
```

Listing 7: Resultado de ejecutar el comando `print_reachable_states` para el modelo inicial

El resultado indica que hay 13596 estado alcanzables y que hay $2,01282e + 08$ (es decir, 201.282.000) combinaciones posibles con la especificación del modelo final. El `system diameter = 34`, indica que todo estado alcanzable desde el estado inicial puede alcanzarse en a lo sumo 34 transiciones.

Ejercicio 7

¿Por qué es importante verificar propiedades de seguridad en sistemas de tiempo real? Dé un ejemplo concreto diferente al de los semáforos.

Dado un sistema/realidad a modelar, se requiere que esta tenga un comportamiento particular esperado. De esta manera, al verificar propiedades, validamos que el sistema

nunca entrará en un estado no deseado o peligroso (safety). Pero también se debe complementar con las propiedades de vitalidad, para asegurarse de que el sistema progrese, y de Fairness, para evitar ejecuciones irreales.

Por ejemplo, un cajero automático solo puede expulsar dinero si el usuario ingresado dispone del mismo.

Explique la diferencia entre CTL y LTL.

La diferencia entre CTL y LTL, es que el primero se define para árboles de ejecución y el último se define para caminos de ejecución.

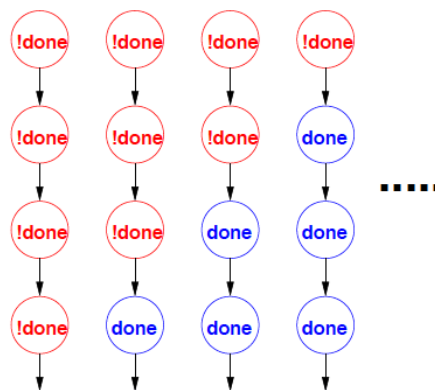


Figura 15: LTL

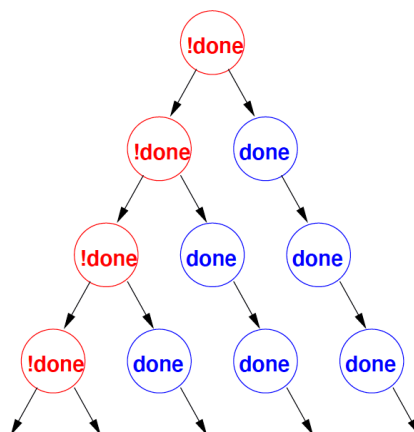


Figura 16: CTL

Además, el LTL permite expresar cómo evolucionan los estados a lo largo del tiempo en una ejecución y el CTL permite combinar cuantificadores de camino con operadores temporales.

¿Cómo modelaría un fallo en el sensor de demanda (ej. sensor siempre activo)?
¿Qué propiedad se violaría?

Podríamos hacer uso de una variable `autosCant` : `Int`, que sea un registro de la cantidad de autos que pasan por la calle del semáforo actual; de esta manera se puede identificar un fallo en caso de que:

- Si el sensor de demanda se mantiene en `false`, y `autosCant` se mantiene en aumento, podremos identificar un fallo del sensor de demanda.
- Si el sensor de demanda se mantiene en `true`, el semáforo de esa misma calle está en verde, y `autosCant` no aumenta, identificando así un fallo en el sensor.

Además, es necesario agregar una nueva variable `sensorFail` : `boolean` que determine si alguna de las dos condiciones mencionadas se cumple. Luego, al momento de la comprobación de demanda, para mantener el semáforo en `green`, la condición se cambiará a que sea `!demand & !sensorFail`.

De esta manera, independientemente de si hay verdaderamente demanda o no para un cruce en particular, si hay falla de sensor, entonces el semáforo no considera extenderse por demanda.

Luego, considerando de que nos estamos refiriendo a las propiedades especificadas para esta actividad, y no las propiedades de *safety*, *liveness* y *fairness*, ninguna propiedad se violaría, siguiendo el modelo completo y el agregando la variable `sensorFail` : `boolean`, debido a que:

1. La propiedad ϕ_1 se mantiene ya que este modelo no permite semáforos en verde en simultáneo. Cuando falla un sensor de un semáforo, el semáforo actúa con su tiempo mínimo siempre.
2. La propiedad ϕ_2 se sigue cumpliendo ya que el semáforo sigue teniendo su comportamiento normal de `green` $\rightarrow$ `yellow` $\rightarrow$ `red` $\rightarrow$ `green`, por lo que `yellow` seguirá tardando T_{yellow} .
3. La propiedad ϕ_3 se mantiene ya que el semáforo sigue teniendo el comportamiento de `green` $\rightarrow$ `yellow` $\rightarrow$ `red` $\rightarrow$ `green` determinado, por lo que eventualmente ocurrirá el cambio para el otro semáforo.
4. La propiedad ϕ_4 se cumple ya que, si no funciona el sensor de demanda, el tiempo verde cumple un tiempo mínimo, por lo que nunca excede el T_M .
5. La propiedad ϕ_5 se sigue manteniendo ya que, aun si falla algún sensor de demanda, entonces el semáforo aún actúa de manera cíclica y por tanto siempre en algún momento algún semáforo estará en verde.